Chat

SSE Chat (Latest Snapshot)

Start a Server-Sent Events (SSE) chat session with the latest snapshot of an agent

Overview

This endpoint initiates a Server-Sent Events (SSE) streaming chat session with the latest snapshot of a specified agent. It automatically uses the most recent snapshot of the agent, ensuring you're always chatting with the up-to-date configuration without needing to specify a specific snapshot ID.

Path Parameters

`agent_id` `string` `required`

The unique identifier of the agent whose latest snapshot you want to chat with

Query Parameters

`end_user_id` `string` `required`

Identifier for the end user initiating the chat session

`chat_id` `string`

Optional chat ID to resume an existing chat session. If not provided, a new chat will be created.

`user_message` `string`

Message to pass to the agent

Authentication

This endpoint requires OAuth2 Bearer token authentication. Include the token in the Authorization header:

```
Authorization: Bearer your_access_token
```

Response

The endpoint returns a Server-Sent Events stream with the same format as the snapshot-specific endpoint. Each event contains a JSON-serialized `LeapingResponse` object sent as data-only SSE events.

Response Format

Each SSE event contains a JSON-serialized `LeapingResponse` with the following structure:

`type` `string` `required`

Discriminator field indicating the response type. Common values include:

- `chat_message` : Agent or human messages
- `turn` : Turn management (whose turn to speak)
- `error` : Error messages
- `transition` : Stage transitions in conversation flow
- `start` : Conversation initialization
- `end` : Conversation completion with summary and results
- `function` : Function call execution
- `kb` : Knowledge base lookup results

`timestamp` `number` `required`

Unix timestamp when the response was generated

For detailed field descriptions of each response type, see the **Chat with Snapshot (SSE)** documentation.

Usage Examples

JavaScript/Browser

```
const eventSource = new EventSource(
  '/chat/agent/your-agent-id/latest?end_user_id=user123&user_message=Hello',
  {
    headers: {
      'Authorization': 'Bearer your_access_token'
    }
  }
```

··· See all 41 lines

Python with httpx-sse

```
import httpx
import json
import asyncio
from httpx_sse import aconnect_sse
from typing import Optional, Dict, Any
from datetime import datetime

```

··· See all 270 lines

cURL

```
curl -X POST \
  "https://api.leaping.ai/v1/chat/agent/your-agent-id/latest?end_user_id=user123" \
  -H "Authorization: Bearer your_access_token" \
  -H "Accept: text/event-stream"
```

Advantages of Latest Snapshot

Using the latest snapshot endpoint provides several benefits:

- **Always Current**: Automatically uses the most recent agent configuration
- **Simplified Integration**: No need to track or manage snapshot IDs
- **Development Friendly**: Ideal for development and testing environments
- **Automatic Updates**: Agent improvements are immediately available

Caching Behavior

The system implements intelligent caching for performance:

1. **Redis Cache**: Latest snapshots are cached in Redis for fast retrieval
2. **Cache Miss Handling**: Falls back to Supabase database when cache is unavailable
3. **Automatic Updates**: Cache is updated when new snapshots are created

Resuming Chats

When resuming a chat with a latest snapshot endpoint, the system validates that the chat was originally created with the same agent:

```
// Resume existing chat
const eventSource = new EventSource(
  '/chat/agent/your-agent-id/latest?end_user_id=user123&chat_id=existing-chat-id'
);
```

Important: You can only resume chats that were originally started with the same agent ID.

Error Handling

Common error scenarios specific to this endpoint:

- **404**: Agent not found or no snapshots available
- **400**: Cannot resume chat with different agent (when using chat_id)
- **401**: Invalid or missing authentication token
- **403**: Insufficient permissions to access the agent

Example error response:

```
{
  "type": "error",
  "message": "Cannot resume chat with different agent.",
  "timestamp": 1704067200.123
}
```

Best Practices

- Use this endpoint in production when you want the latest agent behavior
- Implement proper error handling for agent compatibility issues
- Consider using specific snapshot endpoints for version-controlled environments
- Monitor agent updates that might affect existing chat sessions
- Store both agent_id and the actual snapshot_id used for audit purposes

Comparison with Snapshot Endpoint

Feature	Latest Snapshot	Specific Snapshot
Version Control	Always latest	Specific version
Consistency	May change between calls	Guaranteed consistent
Development	Ideal for testing	Better for production
Use Case	Dynamic environments	Stable, versioned deployments

Ask a question...

Powered by mintlify